Food Delivery Management Platform

Software Requirements Specification (SRS) for UCS503P

Submitted to: Dr. Paramveer Sidhu

Thapar Institute of Engineering and Technology

Sumit Keshri (COE)

1. Abstract

Food Delivery Management Platform is a web-based system connecting customers, restaurants, delivery partners, and administrators through a unified food-ordering and delivery workflow. Customers can browse restaurants, view menus, add food items to a cart, place orders, make payments, and track delivery status. Restaurants can manage menus and orders, delivery partners can manage assigned deliveries, and administrators can monitor users, restaurants, payments, complaints, and platform performance.

The project focuses on a reliable, scalable, and trackable workflow from food selection to final delivery.

2. Problem Statement

Traditional food ordering through phone calls or separate restaurant systems creates fragmentation, limited order transparency, restaurant management difficulties, delivery coordination problems, and poor operational visibility. The platform provides one integrated system for food ordering and delivery management.

3. Higher-Order Goal

Improve the efficiency and transparency of food ordering and delivery by connecting customers, restaurants, delivery partners, and administrators through a common digital workflow. The engineering objective is a deployable full-stack web application with role-based access, order management, payment processing, delivery tracking, testing, and measurable performance.

4. Time-to-Value

Iteration 1: registration/login, restaurant listing, menu browsing, cart, order placement, and basic status.

Iteration 2: restaurant dashboard, delivery partner dashboard, payment integration, and order assignment.

Later iterations: real-time tracking, notifications, reviews, analytics, recommendations, and advanced search.

5. Purpose

This SRS defines the functional and non-functional requirements of the Food Delivery Management Platform and acts as a reference for system design, database design, implementation, testing, evaluation, and deployment.

6. Scope

In Scope: customer authentication, restaurant management, menu management, restaurant and food search, cart, order placement/cancellation, online payment, order tracking, delivery assignment, delivery updates, reviews, admin dashboard, analytics, and notifications.

Non-Goals: native mobile application in Version 1, advanced fleet optimization, multi-country operation, and fully automated restaurant onboarding.

7. Glossary

Customer — user who browses restaurants and places orders.

Restaurant — food provider registered on the platform.

Delivery Partner — person responsible for collecting and delivering orders.

Admin — system administrator.

Order — customer request containing selected food items.

Cart — temporary collection of food items before checkout.

Menu — food items offered by a restaurant.

Order Status — current stage of an order.

ETA — Estimated Time of Arrival.

8. System Actors

Customer: register/login, browse, search, cart, order, payment, tracking, cancellation, reviews.

Restaurant Staff: manage menu, accept/reject orders, update preparation status, mark items unavailable.

Delivery Partner: login, view assignments, accept deliveries, update status, complete delivery.

Administrator: manage users/restaurants/partners, monitor orders, handle complaints, view analytics, suspend accounts.

9. Core Order Workflow

Customer → Browse Restaurants → Select Restaurant → View Menu → Add to Cart → Checkout → Payment → Order Created → Restaurant Accepts → Food Being Prepared → Delivery Partner Assigned → Picked Up → Out for Delivery → Delivered → Customer Reviews.

10. Functional Requirements

FR-1 Authentication: role-based registration and secure login.

FR-2 Restaurant Management: administrators can create, approve, update, suspend, and remove restaurant accounts.

FR-3 Menu Management: restaurant staff can add, edit, delete, price, upload images, and toggle availability.

FR-4 Restaurant Search: search/filter by name, cuisine, rating, price range, and availability.

FR-5 Food Search: search individual food items.

FR-6 Cart: add/remove items, quantities, subtotal, coupon, delivery charges, final amount.

FR-7 Order Creation: generate a unique order ID and store customer, restaurant, items, price, address, payment status, order status, and timestamps.

FR-8 Restaurant Processing: accept/reject, prepare, and mark ready for pickup.

FR-9 Delivery Assignment: assign eligible delivery partner based on availability, area, and workload.

FR-10 Delivery Tracking: assigned → picked up → out for delivery → delivered.

FR-11 Payment: online payment with payment ID, amount, status, and transaction reference.

FR-12 Cancellation: allow cancellation only during eligible stages.

FR-13 Notifications: order acceptance, preparation, pickup, delivery, completion, and cancellation.

FR-14 Reviews: rating and review after delivery.

FR-15 Admin Dashboard: users, restaurants, orders, revenue, pending/completed/cancelled orders, average delivery time.

11. Database Design

Users(user_id, name, email, password_hash, phone, role)

Restaurants(restaurant_id, owner_id, name, address, cuisine, rating, status)

MenuItems(item_id, restaurant_id, name, description, price, image, availability)

Orders(order_id, customer_id, restaurant_id, delivery_partner_id, total_amount, status, created_at)

OrderItems(order_item_id, order_id, item_id, quantity, price)

Payments(payment_id, order_id, amount, status, transaction_id)

Deliveries(delivery_id, order_id, partner_id, pickup_time, delivery_time, status)

Reviews(review_id, order_id, customer_id, rating, comment)

Notifications(notification_id, user_id, message, timestamp)

12. Web Architecture

Users → React Frontend → REST API → Node.js + Express Backend → PostgreSQL Database.

Backend responsibilities: authentication, authorization, validation, order logic, payment logic, delivery logic, notifications, and API services.

13. Recommended Technology Stack

Frontend: React.js, JavaScript, Tailwind CSS, React Router, Axios.

Backend: Node.js, Express.js, REST API.

Database: PostgreSQL with Prisma or Sequelize.

Authentication: JWT with bcrypt/Argon2.

Real-Time: Socket.IO.

Testing: Jest, Supertest, Postman.

DevOps: Git, GitHub, GitHub Actions, Docker.

Deployment: cloud hosting with managed PostgreSQL.

14. Non-Functional Requirements

Security: secure password hashing, authentication, role-based authorization, HTTPS/TLS, and no storage of sensitive card information.

Performance: target approximately 500 ms response time at the 95th percentile under expected project load; use pagination and indexes; prevent duplicate concurrent orders.

Reliability: idempotency, verified payments, and strict order-state transitions.

Usability: responsive UI, clear order status, transparent pricing, clear errors, and minimal unnecessary steps.

15. Scalability

Use database indexing, pagination, stateless APIs, caching, efficient queries, separated frontend/backend services, and containerized deployment to support growth from a small pilot to a larger platform.

16. Advanced Features

AI food recommendations based on previous orders; smart delivery assignment using distance/availability/workload/ETA; demand prediction using historical data; personalized offers; and restaurant analytics for best-selling items, revenue, peak hours, ratings, and cancellation rate.

17. Testing Strategy

Unit testing for calculation and validation functions; integration testing for Login → API → Database; API testing with Postman; security testing for unauthorized access, invalid tokens, role escalation, injection attempts, and validation; performance testing for concurrent orders.

18. CI/CD Pipeline

Developer → Git Push → GitHub → GitHub Actions → Lint → Unit Tests → Integration Tests → Build → Docker Image → Deployment.

19. Evaluation Metrics

Order Success Rate; Average Delivery Time = Delivered Time − Order Creation Time; Cancellation Rate = Cancelled Orders / Total Orders × 100; API response time; system availability; and payment success rate.

20. Risks and Mitigation

Payment failure — payment verification and retry handling.

Duplicate orders — idempotency keys and server-side validation.

Database failure — backups and recovery procedures.

Unauthorized access — JWT and role-based authorization.

Slow queries — indexes, pagination, and query optimization.

Partner unavailable — reassignment mechanism.

Incorrect status — strict state-transition rules.

Poor mobile experience — responsive UI testing.

API failure — error handling, logging, and monitoring.

21. 12-Week Development Timeline

Weeks 1–2: requirements, SRS, UML, ER diagram, architecture, UI wireframes.

Weeks 3–4: authentication, roles, database, restaurant module.

Weeks 5–6: menu, search, cart, checkout.

Weeks 7–8: order management, restaurant dashboard, delivery module.

Weeks 9–10: payment, notifications, reviews, admin dashboard.

Week 11: testing, security, performance, bug fixing.

Week 12: Docker, CI/CD, deployment, documentation, final presentation.

22. Requirement Traceability

FR-1 Authentication → Auth → TC-AUTH-01

FR-3 Menu Management → Restaurant → TC-MENU-01

FR-6 Cart → Cart → TC-CART-01

FR-7 Order Creation → Order → TC-ORDER-01

FR-10 Delivery Tracking → Delivery → TC-DEL-01

FR-11 Payment → Payment → TC-PAY-01

FR-14 Reviews → Review → TC-REV-01

NFR-SEC-3 Authorization → Security → TC-SEC-03

NFR-PERF-1 API Performance → Backend → TC-PERF-01

23. Recommended Final Project Scope

Implement the core platform with Customer, Restaurant, Delivery Partner, and Admin roles; PostgreSQL; REST APIs; JWT authentication; payment integration; order tracking; automated tests; Docker; CI/CD; and deployment. If time permits, add AI recommendations and smart delivery assignment.

24. Suggested Project Repository Structure

Food-Delivery-Platform/

■■■ frontend/

■ ■■■ components/

■ ■■■ pages/

■ ■■■ services/

■ ■■■ authentication/

■■■ backend/

■ ■■■ controllers/

■ ■■■ routes/

■ ■■■ models/

■ ■■■ middleware/

■ ■■■ services/

■ ■■■ tests/

■■■ database/

■ ■■■ schema/

■ ■■■ migrations/

■■■ docs/

■ ■■■ SRS

■ ■■■ UML

■ ■■■ ER-Diagram

■ ■■■ Architecture

■ ■■■ Test-Plan

■■■ docker/

■■■ .github/

■ ■■■ workflows/

■■■ README.md